

Input e output

Gestione dei file

Come leggere e scrivere file con Python — per salvare e caricare dati.

Perché lavorare con i file?

Quando un programma si chiude, tutto quello che era in memoria scompare. Se vuoi che i dati sopravvivano tra una esecuzione e l'altra (come i punteggi di un gioco, o una lista di nomi), devi **salvarli su file**.

Aprire un file

Per lavorare con un file, lo "apri" con la funzione `open()`. Devi specificare il nome del file e la **modalità** (cosa vuoi fare):

Modalità	Cosa fa
"r"	Legge il file (deve esistere)
"w"	Scriva nel file (lo crea se non esiste, lo svuota se esiste)
"a"	Aggiunge alla fine del file senza cancellare il contenuto

Il modo corretto: il blocco `with`

Usa sempre il costrutto `with` per aprire i file. Garantisce che il file venga chiuso correttamente anche se c'è un errore:

```
with open("esempio.txt", "r") as file:
    contenuto = file.read()
    print(contenuto)
# Il file viene chiuso automaticamente quando esci dal blocco with
```

Leggere un file

Leggere tutto il contenuto in una volta:

```
with open("esempio.txt", "r") as file:
    contenuto = file.read()
    print(contenuto)
```

Leggere riga per riga (utile per file grandi):

```
with open("esempio.txt", "r") as file:
    for riga in file:
        print(riga.strip()) # strip() rimuove il carattere di a capo alla
fine
```

Ottenere tutte le righe come lista:

```
with open("esempio.txt", "r") as file:
    righe = file.readlines() # lista di stringhe, una per riga

print(righe)
```

Scrivere su un file

Creare un nuovo file (o sovrascriverlo):

```
with open("output.txt", "w") as file:
    file.write("Prima riga\n")
    file.write("Seconda riga\n")
    file.write("Terza riga\n")
```

:::caution La modalità `"w"` cancella tutto il contenuto precedente del file. Se vuoi aggiungere senza cancellare, usa `"a"`. :::

Aggiungere righe a un file esistente:

```
with open("output.txt", "a") as file:  
    file.write("Questa riga viene aggiunta alla fine\n")
```

Gestire il caso in cui il file non esiste

Se provi ad aprire un file che non esiste in modalità `"r"`, Python dà un errore. Usa `try/except` per gestirlo:

```
try:  
    with open("dati.txt", "r") as file:  
        contenuto = file.read()  
        print(contenuto)  
except FileNotFoundError:  
    print("Il file non esiste ancora.")
```

Esempio pratico: salvare e caricare una lista

```
def salva_studenti(studenti, nome_file):
    """Salva la lista degli studenti su file."""
    with open(nome_file, "w") as file:
        for nome, voto in studenti:
            file.write(f"{nome},{voto}\n")

def carica_studenti(nome_file):
    """Carica la lista degli studenti dal file."""
    studenti = []
    try:
        with open(nome_file, "r") as file:
            for riga in file:
                parti = riga.strip().split(",")
                if len(parti) == 2:
                    nome = parti[0]
                    voto = float(parti[1])
                    studenti.append((nome, voto))
    except FileNotFoundError:
        print("File non trovato, parto con lista vuota.")
    return studenti

# Salva i dati su file
studenti = [("Alice", 8.5), ("Bob", 7.0), ("Carlo", 9.2)]
salva_studenti(studenti, "studenti.csv")

# Ricarica i dati dal file
caricati = carica_studenti("studenti.csv")
for nome, voto in caricati:
    print(f"{nome}: {voto}")
```

Il file `studenti.csv` conterrà:

```
Alice,8.5
Bob,7.0
Carlo,9.2
```

Formattazione dell'output

Come presentare i dati in modo leggibile e ordinato.

Perché formattare l'output?

Stampare dati grezzi funziona, ma non è sempre leggibile. Formattare l'output significa presentare le informazioni in modo chiaro: numeri con il numero giusto di cifre decimali, testo allineato in colonne ordinate.

Le f-string: il metodo consigliato

Le **f-string** (disponibili da Python 3.6) sono il modo più comodo e leggibile per costruire stringhe con variabili. Si scrivono mettendo **f** prima delle virgolette:

```
nome = "Alice"
eta = 16
print(f"Mi chiamo {nome} e ho {eta} anni.")
# Mi chiamo Alice e ho 16 anni.
```

Dentro le parentesi graffe puoi scrivere qualsiasi espressione Python:

```
a, b = 5, 3
print(f"{a} + {b} = {a + b}")
print(f"Il quadrato di {a} è {a**2}")
```

Controllare i decimali

Per i numeri decimali puoi specificare quante cifre mostrare dopo la virgola:

```
pi = 3.14159265
print(f"{pi:.2f}")    # 3.14    - 2 cifre decimali
print(f"{pi:.4f}")    # 3.1416  - 4 cifre decimali
print(f"{pi:.0f}")    # 3        - nessuna cifra decimale
```

La sintassi è `{valore:.Nf}` dove **N** è il numero di cifre decimali.

Percentuali

```
percentuale = 0.75
print(f"{percentuale:.0%}")    # 75%
print(f"{percentuale:.2%}")    # 75.00%
```

Python moltiplica per 100 e aggiunge il simbolo `%` automaticamente.

Separatore delle migliaia

Per i numeri grandi, puoi aggiungere la virgola come separatore:

```
numero = 1234567
print(f"{numero:,}")    # 1,234,567
```

Allineare il testo in colonne

Quando vuoi creare tabelle con colonne ordinate, puoi specificare la larghezza e l'allineamento:

```
print(f"{'Sinistra':<15}|")    # allineato a sinistra – 15 caratteri
print(f"{'Centro':^15}|")      # centrato
print(f"{'Destra':>15}|")      # allineato a destra
```

Output:

```
Sinistra      |
      Centro  |
           Destra|
```

Opzioni di `print()`

La funzione `print()` ha due opzioni utili:

sep — il testo che viene messo tra gli argomenti (default: uno spazio):

```
print("mela", "banana", "uva")           # mela banana uva
print("mela", "banana", "uva", sep=", ")  # mela, banana, uva
print("mela", "banana", "uva", sep=" | ") # mela | banana | uva
```

end — il testo aggiunto alla fine (default: va a capo `\n`):

```
print("Ciao", end=" ")
print("mondo!")           # Ciao mondo!  – sulla stessa riga
```

Utile per stampare più elementi sulla stessa riga in un ciclo:

```
for i in range(5):
    print(i, end=" ")
print() # va a capo alla fine
# Output: 0 1 2 3 4
```

Esempio: una tabella formattata

```
studenti = [  
    ("Alice", 16, 8.5),  
    ("Bob", 15, 7.0),  
    ("Carlo", 17, 9.2),  
]  
  
# Intestazione della tabella  
print(f"{'Nome':<10} {'Età':>5} {'Media':>8}")  
print("-" * 25)  
  
# Righe della tabella  
for nome, eta, media in studenti:  
    print(f"{'nome':<10} {'eta':>5} {'media':>8.1f}")
```

Output:

Nome	Età	Media
Alice	16	8.5
Bob	15	7.0
Carlo	17	9.2

Input utente

Come chiedere informazioni all'utente durante l'esecuzione del programma.

Perché l'input è importante?

Un programma che fa sempre le stesse cose con gli stessi dati è limitato. Con l'**input utente**, il programma può fare domande e comportarsi in modo diverso in base alle risposte. Questo è il primo passo per creare programmi davvero interattivi.

La funzione `input()`

`input()` mostra un messaggio e poi si mette in attesa: il programma si ferma finché l'utente non scrive qualcosa e preme Invio.

```
nome = input("Come ti chiami? ")
print(f"Ciao, {nome}!")
```

Esempio di esecuzione:

```
Come ti chiami? Alice
Ciao, Alice!
```

Il testo dentro le parentesi di `input()` è il messaggio che viene mostrato all'utente.

Attenzione: `input()` restituisce sempre testo

Anche se l'utente digita un numero, `input()` lo tratta sempre come **testo** (stringa). Se vuoi fare calcoli, devi convertire:

```
# Questo NON funziona per fare calcoli:
eta_testo = input("Quanti anni hai? ")
# eta_testo è una stringa, es. "16" – non il numero 16!

# Converti con int() per avere un numero intero:
eta = int(input("Quanti anni hai? "))
print(f"Tra 10 anni avrai {eta + 10} anni.")

# Converti con float() per numeri decimali:
altezza = float(input("Altezza in metri: "))
print(f"Altezza: {altezza} m")
```

Cosa succede se l'utente sbaglia?

Se l'utente digita "pippo" invece di un numero e tu usi `int()`, il programma crasha con un errore.

Per gestire questo caso elegantemente, usa `try/except` :

```
try:
    numero = int(input("Inserisci un numero: "))
    print(f"Hai inserito: {numero}")
except ValueError:
    print("Quello non era un numero! Riprova.")
```

Continuare a chiedere finché la risposta è valida

Un pattern molto comune: ripetere la domanda finché l'utente non inserisce un valore accettabile.

```
while True:
    try:
        eta = int(input("Inserisci la tua età: "))
        if eta < 0 or eta > 120:
            print("Età non valida. Inserisci un numero tra 0 e 120.")
            continue # riparte dall'inizio del ciclo
        break # esce dal ciclo se tutto è ok
    except ValueError:
        print("Scrivi un numero, non del testo.")

print(f"La tua età è: {eta}")
```

Esempio pratico: calcolatrice

```
print("Calcolatrice semplice")
print("-" * 20)

try:
    a = float(input("Primo numero: "))
    operazione = input("Operazione (+, -, *, /): ")
    b = float(input("Secondo numero: "))

    if operazione == "+":
        risultato = a + b
    elif operazione == "-":
        risultato = a - b
    elif operazione == "*":
        risultato = a * b
    elif operazione == "/":
        if b == 0:
            print("Errore: non si divide per zero!")
        else:
            risultato = a / b
    else:
        print("Operazione non riconosciuta.")

    print(f"Risultato: {a} {operazione} {b} = {risultato}")

except ValueError:
    print("Inserisci solo numeri validi.")
```